Open in app ↗

Medium Search Write

How Angular 19 HMR works internally (Vite vs Webpack)

Piyali Das 3 min read · Apr 7, 2026

1

Great question — this is where Angular 19 made a **major architectural shift.**

Angular is no longer just "Webpack-based." It now uses a **Vite-powered dev pipeline (via esbuild)** under the hood for `ng serve`.

```
| Layer          | Old Angular            | Modern Angular (19+)       |
| -------------- | ---------------------- | -------------------------- |
| **Dev Server** | Webpack Dev Server     | **Vite**                   |
| **Bundler**    | Webpack                | **esbuild** (dev)          |
| **Compiler**   | tsc + Angular compiler | esbuild + Angular compiler |
```

👉 Vite replaces:

- Webpack Dev Server
- Part of Webpack's dev pipeline

- Custom Angular CLI dev server logic

🧩 Mental Model

```
OLD WORLD
---------
Webpack = Bundler + Dev Server + HMR

NEW WORLD
---------
Vite = Dev Server + HMR
esbuild = Bundler / Transformer
```

👉 So:

> **esbuild replaces bundling**
>
> **Vite replaces dev server + HMR engine**

⚙️ Replace Angular CLI with pure Vite + esbuild

⚠️ Important truth first:

👉 Angular is **not fully Vite-native yet**, because it needs:

- Angular Compiler (Ivy)
- Template processing
- Decorators

So we use a **custom Vite plugin for Angular**

🧠 Big Picture

Before Angular 19

- Dev server: **Webpack Dev Server**
- HMR: **Webpack HMR plugin**
- Rebuild speed: 🐢 slower

Angular 19+

- Dev server: **Vite (wrapped by Angular CLI)**
- Bundler: **esbuild (fast)**
- HMR: **Vite-style module graph updates**
- Rebuild speed: ⚡ very fast

⚙️ Angular 19 Dev Pipeline (Internal Flow)

```
Angular CLI
   ↓
Vite Dev Server
   ↓
esbuild (transforms TS → JS)
   ↓
Browser (ES Modules)
   ↓
WebSocket (HMR channel)
```

🔌 Step-by-Step: How HMR Works Internally

1. File Change Detection

When you edit a file:

```
users.component.html
```

👉 Angular CLI watches filesystem

2. Incremental Build (esbuild)

Instead of rebuilding everything:

```
Only changed module is recompiled
```

👉 This is why it's FAST ⚡

3. Vite Module Graph

Vite maintains a **dependency graph like this:**

```
AppComponent
 ├── UsersComponent
 │      ├── users.component.html
 │      └── users.service.ts
 └── HeaderComponent
```

👉 It knows exactly:

- What depends on what
- What needs to be updated

4. WebSocket Push

Dev server sends update:

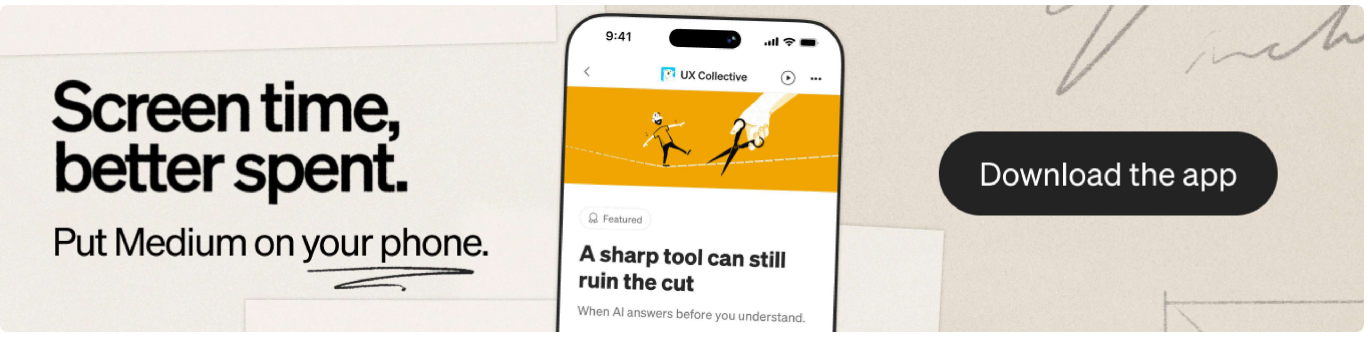
```
{
  "type": "update",
  "path": "/src/app/users.component.html"
}
```

👉 This is what you see as:

```
client → websocket (Pending)
```

5. Browser Applies HMR

Now Angular runtime decides:



✅ Case 1: Template / Style change

```
Replace template in component instance
```

✓ No reload
✓ State preserved

❌ Case 2: TypeScript change

```
Module invalidated → full reload
```

👉 Because Angular:

- Compiles TS → JS
- Needs re-bootstrap for logic changes

⚡ Vite vs Webpack (Core Difference)

🟣 Webpack HMR (Old Angular)

```
Change → Rebuild bundle → Push updated chunk → Patch runtime
```

❌ Heavy
❌ Slower
❌ Complex dependency resolution

🟢 Vite HMR (Angular 19)

```
Change → Transform single module → Push via ESM → Replace instantly
```

✓ No bundling step
✓ Native ES modules
✓ Much faster

🔥 Key Technical Differences

```
| Feature          | Webpack      | Vite (Angular 19)   |
| --------------- | ----------- | ------------------ |
| Bundling         | Required     | Not required (dev) |
| Module system    | Custom       | Native ES Modules  |
| Rebuild speed    | Slow         | ⚡ Fast             |
| HMR granularity | Chunk-based | Module-based       |
| Dev startup      | Slow         | Instant            |
```

🧪 Example: Real HMR Behavior

Change HTML

```
<h1>{{title}}</h1>
```

👉 Vite sends update → Angular swaps template
✅ Instant update

Change CSS

```
h1 { color: blue; }
```

👉 Injected into `<style>` tag
✅ No reload

Change TS

```
title = "New Title";
```

👉 Module invalid → full reload
❌ Not true HMR yet

⚠️ Why Angular Can't Fully HMR TS (Important)

Unlike React/Vue:

Angular has:

- Dependency Injection
- Zones / Signals
- Decorators (`@Component`)

👉 **Changing TS can affect:**

- Component lifecycle

- Dependency graph
- Metadata

So Angular plays safe → reloads app

🚀 Hidden Detail (Most People Don't Know)

Angular 19 uses:

👉 **esbuild for TS transpilation**
👉 **Vite only for dev server + HMR layer**

So it's actually a **hybrid architecture**:

```
esbuild (compiler) + Vite (server) + Angular runtime (patching)
```

🧩 Where Webpack Still Exists

- Production builds (`ng build`) may still use optimized pipeline
- Legacy builders still available

👉 But direction is:
Fully Vite-based Angular in future

✅ Final Summary

👉 Angular 19 HMR works like this:

1. File changes detected
2. esbuild recompiles only that module
3. Vite sends update via WebSocket
4. Angular runtime:

- Patches template/style ✅
- Reloads app for TS ❌

💡 Real Insight

> Angular 19 HMR is **partial but fast**
> It's a huge step toward **React/Vite-level DX**, but not fully there yet.

Angular | Coding | UI | Technology

Written by Piyali Das

787 followers · 9 following

Edit profile

12 + yrs | Angular | NGRX | RxJS | https://www.linkedin.com/in/piyalidas10/
https://stackblitz.com/@piyalidas10 | https://github.com/piyalidas10

No responses yet

Piyali Das

What are your thoughts?